

Scripting Language

- Combine existing components together
- Not a full program and it requires a **host environment** where it is usually interpreted
- Limited support for UI design, graphics and data types
- **Web Scripts** may run on the **server** or **client** side
- Server side host environment - web server
- Client side host environment - web browser (most common)

Client Side we script

- event handling, animation, interactive content, dynamic generation of web content, partial loading of web content, client side validation
- Script is **integrated with HTML**
- Responds to user interaction and other events
- Has access to objects in host environment (browser window, browser history, cookies, Web document components)

JavaScript

- **Most widely used client-side** scripting language, though it can also be used for server-side scripting
- Evolved into a **high level language**
 - paradigms: event-driven, imperative, object-oriented (prototype-based), functional
 - weakly typed, dynamically typed
 - JIT (Just-In-Time) compiled by JavaScript engines
 - use of transpilers for backward compatibility
 - treats functions as first-class objects
- Conceptualized by **Brendan Eich** in **1995**
- JavaScript standards are defined by **ECMA International (European Computer Manufacturers Association)**, specifically by the committee called **TC39 (Technical Committee 39)**.
- The official name of the JavaScript standard is **ECMAScript**, latest if **ECMAScript 16 (Oct 2025)**
- JavaScript is an implementation of this specification

Other Specifications Related to JavaScript

Extends Javascript but isn't Javascript itself. Usually compiled or transpiled into standard Javascript

1. **TypeScript**
 - A superset of javascript
2. **ActionScript (Adobe)**
3. **CoffeeScript**

JavaScript Scripting Engines

Programs that interpret and execute JavaScript code — either in browsers or standalone environments (like Node.js).

1. SpiderMonkey(Firefox)
2. V8(Chrome, Opera, Brave, Microsoft Edge, [Node.js](#))
3. JavaScriptCore(Safari and other iOS browsers)

JavaScript in HTML Documents

- JS can be specified anywhere in the HTML document
- Javascript can be the following:
 - External Script

```
<script src='/path/to/script.js'></script>
```

- Internal Script (Can be in Head or Body)

```
<head>
  <script>
    // top-level code
    var max = 10;
    for (i = 0; i < max; i++) {
      console.log('hello');
    }
  </script>
</head>
```

- Inline Script

```
<body>
  <h1>Execution of JavaScript</h1>
  <p><button onclick='alert("You clicked me!");'>ClickMe!</button></p>
</body>
```

- Noscript Element (Specifies if javascript isn't enabled)

```
<body>
  <noscript>
    JavaScript learner, go get a modern browser. ;- )
  </noscript>
</body>
```

- Browser console runs JS in **interpreted mode** on the **main thread**
 - Exceptions are web workers
 - console.log() displays output

- Browser reads and executes your JS code **directly instead of compiling** it into a standalone program first

Variable Declarations

- **Var**
 - Global
 - Default when there is no keyword
- **Let**
 - Block scoped
 - Cannot be redeclared in the same scope
- **Const**
 - Constant reference to a value
 - Block scoped
 - Cannot be redeclared or redefined

Primitive Data Types

Primitive types are **the most basic data values** in JavaScript.

They are **not objects** and have **no methods**—but JavaScript can temporarily wrap them with objects when needed.

- number (integer and decimal) , string, boolean, bigint, null, and undefined.

Wrapper Objects

JavaScript provides object “wrappers” for primitives so they can behave like objects temporarily. These wrappers give access to methods and properties

- Number (integer and decimal) , String, Boolean, BigInt

Object Types

Everything else in JavaScript is an **object** — meaning it can hold properties and methods.

- Object, Function, Array, Math, Map , Set, Date , JSON , RegExp , Error , Promise

Booleans

- "falsy" values vs "truthy" values
- Equality

Strings

- Delimited by single quotes, double quotes or back-tick
- Back-tick is useful for template literals

User Defined Objects

- In JavaScript, an **object** is a collection of **properties**.
- Each **property** has a **key (name)** and a **value**.
- That value **can be any type** — a number, string, another object, or even a **function** (which is then called a **method**).

For Example:

```
let person = {  
  name: "Jec",  
  age: 20,  
  greet: function() {  
    console.log("Hello, my name is " + this.name);  
  }  
};
```

```
person.greet(); // "Hello, my name is Jec"
```

- You can add properties **after** an object is created — JavaScript objects are **dynamic**.

- **Object Definition Methods**

- **Object Type Constructor**

```
let obj = new Object();  
obj.name = "Jec";  
obj.age = 20;
```

```
console.log(obj.name); // "Jec"
```

- **Object Literal Syntax**

```
let obj = {  
  name: "Jec",  
  age: 20  
};
```

- **Object Construction Function**

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
  this.greet = function() {  
    console.log("Hi, I'm " + this.name);  
  };  
}
```

```
let p1 = new Person("Jec", 20);  
p1.greet(); // "Hi, I'm Jec"
```

- **Object.create() Method**

```
let animal = {  
  eat: function() {  
    console.log("Animal is eating");  
  }  
};
```

```
let dog = Object.create(animal);
dog.bark = function() {
  console.log("Woof!");
};

dog.eat(); // "Animal is eating" (inherited)
dog.bark(); // "Woof!" (own property)
```

JavaScript APIs

- JavaScript can also access **Web APIs** that let it interact with the browser and the web page. These APIs aren't part of "core JS" but are provided by the **browser**.
- **Browser Object Model (BOM) API**
 - Non-standard
 - Window, Navigator, Screen, Location, History
- Others may be Canvas, Fetch, Geolocation, IndexedDB, Local Storage, Push, Service Worker, Websockets, Web Workers, etc.

Document Object Model (DOM) API

- **DOM Tree**
 - Representation of a parsed HTML document
 - Accessed through the Document object property of the Window object
 - Basic DOM Interfaces - Document, Node, Element, Event. Text, Attr
 - innerHTML outerHTML innerText outerText, head, body, anchors, forms, images are available but **not part of the API**

DOM Functionalities

1. Construct / Copy Nodes

These methods **create new elements** or nodes in memory (not yet visible on the page).

Function	Description	Example
<code>document.createElement()</code>	Creates a new HTML element.	<pre>let div = document.createElement("div");</pre>

<code>document.createTextNode()</code>	Creates a text node (plain text).	<pre>let text = document.createTextNode("Hello World");</pre>
<code>document.createAttribute()</code>	Creates a new attribute.	<pre>let attr = document.createAttribute("id"); attr.value = "main";</pre>

■ Example:

```
let p = document.createElement("p");
let text = document.createTextNode("Hi there!");
p.appendChild(text);
document.body.appendChild(p);
```

🔍 2. Retrieve References to Nodes

Used to **get** existing elements from the document.

Function	Description	Example
<code>getElementById()</code>	Finds a single element by its <code>id</code> .	<code>document.getElementById("title")</code>
<code>getElementsByTagName()</code>	Returns all elements with a given tag name.	<code>document.getElementsByTagName("p")</code>
<code>querySelector()</code>	Returns the first element matching a CSS selector.	<code>document.querySelector(".header")</code>
<code>querySelectorAll()</code>	Returns <i>all</i> matching elements.	<code>document.querySelectorAll("div.card")</code>

🌳 3. Traverse the DOM Tree

Lets you **navigate** through parent-child-sibling relationships.

Property	Description
----------	-------------

<code>parentNode / parentElement</code>	Move one level up (to the parent).
<code>childNodes / children</code>	Access child elements.
<code>firstChild / lastChild</code>	Get the first/last child (includes text nodes).
<code>firstElementChild / lastElementChild</code>	Get the first/last <i>element</i> child.
<code>nextSibling / previousSibling</code>	Move sideways to other nodes on the same level.

Example:

```
let list = document.getElementById("menu");
console.log(list.firstElementChild.textContent); // logs text of first
<li>
```

4. Manipulate the DOM Tree

Lets you **add, remove, or replace** nodes dynamically.

Function	Description
<code>appendChild</code> ()	Adds a new child element.
<code>removeChild</code> ()	Removes a specific child element.
<code>replaceChild</code> d()	Replaces one child with another.

Example:

```
let newItem = document.createElement("li");
newItem.textContent = "New Menu Item";
menu.appendChild(newItem); // adds to list
```

5. Event Handling

Used to make pages **interactive** (respond to clicks, input, etc.).

Function	Description
<code>addEventListener()</code>	Adds an event handler (e.g., click, input).
<code>removeEventListener()</code>	Removes an event handler.

Example:

```
button.addEventListener("click", () => alert("Button clicked!"));
```

6. Styles and CSS

Control or read CSS rules dynamically.

Property / Method	Description
<code>styleSheets</code>	Access stylesheets in the document.
<code>cssRules</code>	Access the CSS rules in a stylesheet.
<code>insertRule()</code>	Add a CSS rule dynamically.
<code>getComputedStyle()</code>	Get the final style applied to an element.

Example:

```
let btn = document.getElementById("submit");
let style = getComputedStyle(btn);
console.log(style.backgroundColor);
```

Event Handling

Examples of events:

- **click** → when a button is clicked
- **keydown** → when a key is pressed
- **load** → when the page finishes loading
- **input** → when text in a textbox changes

Event Registration Methods

1. **Inline Event Handler** (Non-standard, older method)

```
<button onclick="sayHello()">Click Me</button>
```

```
<script>
function sayHello() {
  alert("Hello!");
}
</script>
```

2. **Assigning the on<event> Property in JavaScript**

```
<button id="btn">Click Me</button>
```

```
<script>
const btn = document.getElementById("btn");

btn.onclick = function() {
  alert("Button clicked!");
};

// Remove the event later:
btn.onclick = null;
</script>
```

3. **Using **addEventListener()** (Standard, Modern Way)**

```
<button id="btn">Click Me</button>
```

```
<script>
const btn = document.getElementById("btn");

function greet() {
  alert("Hello, SLU student!");
}

btn.addEventListener("click", greet);
```

```
// You can add another handler for the same event:
btn.addEventListener("click", () => console.log("Button was clicked again!"));

// To remove the handler:
btn.removeEventListener("click", greet);
</script>
```

Dispatching Events

```
<button id="btn">Click Me</button>

<script>
const btn = document.getElementById("btn");

// Add an event listener (normal)
btn.addEventListener("click", () => {
  console.log("Button was clicked!");
});

// Create an event object
const event = new Event("click");

// Dispatch (trigger) it manually
btn.dispatchEvent(event);
</script>
```

Custom Events

```
<div id="box"></div>

<script>
// Listen for a custom event
const box = document.getElementById("box");
box.addEventListener("explode", (e) => {
  console.log("Boom! 💣 Power:", e.detail.power);
});

// Create a custom event with extra data
const customEvent = new CustomEvent("explode", {
  detail: { power: 100 } // custom data you can pass
});

// Dispatch it manually
box.dispatchEvent(customEvent);
</script>
```

JavaScript Execution Environment

- **Browser** - Parse HTML, evaluate CSS, load images, download and execute JS, render the page
- **Types of JavaScript Code**
 - **Top-Level Code**
 - Code that is not inside any function or class
 - Executes immediately when the script is loaded
 - **Function Code**
 - Code inside functions
 - Executes only when a function is called

```
<script>  
console.log("1. Top-level code"); // runs immediately
```

```
function sayHello() {  
  console.log("2. Function code"); // runs only when called  
}
```

```
sayHello(); // invoking the function  
</script>
```

Web Document Parsing

- A preload scanner fetches resources like CSS, JS and fonts in the background while the main thread continues to build the DOM
- **CSSOM (CSS Object Model)**
 - A structure describing styles for each element
 - This does not block HTML but may block scripts (due to the effect of styles on elements / DOM objects).
- Regular scripts are downloaded as they are encountered and executed.
- Background processes (e.g., code in web workers) will NOT block HTML.
- The DOMContentLoaded event is triggered.
- When the CSSOM tree is completed, the DOM tree and CSSOM tree are combined to build the **render tree**
- The **load event** is triggered after the entire page is rendered.

Execution Environment – Script Loading Behavior

- **Regular**
 - The browser **stops parsing HTML** when it reaches this script
 - It **downloads** and **executes** the script immediately.
 - Only after execution finishes does it resume parsing HTML.
- **Defer**
 - `<script src="..." defer></script>`

- Script is **downloaded asynchronously** (in parallel) while the HTML continues parsing.
- Script execution happens **after** the DOM is fully built (parsed), but **before** the `DOMContentLoaded` event fires.
- The order of `defer` scripts is preserved (executed in order).
- **Async**
 - Script is **downloaded asynchronously** like `defer`, but executes **as soon as it's fully downloaded** — even if the HTML is still being parsed.
 - The order of execution is **not guaranteed** (depends on download completion).
 - Before the **Load event** is fired